

Revision Question Alternative Set 4

Question 1

- a) Write a query to show the total number of transactions made by each customer.
- Use the COUNT() window function partitioned by customer_id.
 - Join the customers table with the transactions table.
 - The output should include customer_id, first_name, and the transaction count (as total_transactions_count).
- b) Write a query to find all products that have a stock_quantity greater than the average stock quantity of all products.
- Use a subquery to calculate the average stock_quantity.
 - The output should include product_name and stock_quantity.
- c) Write a query to list all transaction items and the category of the product they belong to.
- Use a JOIN to connect transaction_items and products.
 - The output should include transaction_id, product_name, and category.
- d) Write a query to categorize transactions based on their total_amount.
- Use a CASE statement to create a column priority_level:
 - 'High Priority' for amounts greater than 20.00.
 - 'Standard' for amounts between 10.00 and 20.00.
 - 'Low Priority' for amounts below 10.00, else 'Zero Value'.
 - The output should include transaction_id, total_amount, and priority_level.

- e) Write a query to calculate the running total of revenue (using total_amount) across all transactions.
- Use the SUM() window function ordered by transaction_id.
 - The output should include transaction_id, total_amount, and the running revenue (as cumulative_revenue)

Question 2

- a) Write a query to calculate the average price of products within each category.
- Use the AVG() window function partitioned by category.
 - The output should include product_id, product_name, category, and the category average price (as avg_category_price).
- b) Write a query to find the names of all products that have never been part of a 'Completed' transaction.
- Use a subquery to find product_ids associated with the 'Completed' status in the transactions table.
 - The output should include product_name.
- c) Write a query to list all transactions along with the city and country of the customer.
- Use a JOIN to connect transactions and customers.
 - The output should include transaction_id, city, and country.
- d) Write a query to categorize inventory urgency.
- Use a CASE statement:
 - 'Critical' if stock_quantity is less than 70.

- 'Stable' if stock_quantity is 70 or more.
 - The output should include product_name, stock_quantity, and urgency_status.
- e) Write a query to find the smallest and largest total_amount recorded for each transaction status.
- Use MIN() and MAX() functions grouped by status.
 - The output should include status, min_amount, and max_amount.